CMPH 2.0: Practical Extensions to Minimal Perfect Hashing

Compilation, Ordering, and Context

Reini Urban

Independent Researcher
github.com/rurban/cmph

April 2026

Abstract. The CMPH library [1] has provided practical implementations of minimal perfect hash functions (MPHFs) since 2004, covering algorithms including BMZ, CHM, BRZ, FCH, BDZ, and CHD. Releases 2.0 through 2.1 introduce several practically significant extensions: compilation of MPHFs to standalone C code for embedding in applications without runtime library dependencies; support for multiple hash functions (wyhash, CRC32, DJB2, FNV, SDBM) in addition to Jenkins; and the ability to generate an ordering table mapping output indices to input key positions, together with a reordered keyfile. This paper describes these extensions, discusses an important subtlety — that an MPHF without ordering data is smaller than one that preserves key order — and places the CMPH algorithms in the context of the significant advances in the field since 2009, as surveyed by Lehmann et al. [2], which covers PTHash, RecSplit, SIMDRecSplit, ShockHash, MorphisHash, and CONSENSUS.

1 Introduction

A *perfect hash function* (PHF) for a key set $S \subseteq U$ of n keys maps each key to a unique value in $[0, m-1]$ with no collisions. When $m = n$, the function is *minimal* (MPHF), representing a bijection between S and the first n non-negative integers. MPHFs are widely used for space-efficient static dictionaries, keyword recognition in compilers, URL indexing in search engines, and bioinformatics.

The CMPH library [1] was created at the Federal University of Minas Gerais by Davi de Castro Reis, Djamal Belazzougui, Fabiano C. Botelho, and Nivio Ziviani, with continuing maintenance by the present author. It provides a portable C library and command-line tool supporting multiple MPHF algorithms with a unified API for creation, serialization, and lookup.

The period from 2009 to the present has seen dramatic progress in MPHF research [2]. Modern algorithms approach the theoretical space lower bound of $\log_2 e \approx 1.443$ bits per key [3] — a bound that seemed distant when CMPH was first released. CMPH's CHD algorithm [4] already achieved near-practical-optimal space (2.07 bits/key) at the time; the question is how the library's practical tooling fits into this modern landscape.

Releases 2.0 through 2.1 of CMPH focus not on new core algorithms but on *practical enhancements* that increase the library's utility in embedded, compiled, and data-processing contexts:

1. **Compilation to standalone C** (`cmph -C`): serialize an MPHF as a self-contained C source file that can be linked into any C application, eliminating the runtime library dependency and startup deserialization overhead.
2. **Expanded hash function support**: wyhash [5], CRC32, DJB2, FNV, and SDBM, selectable with the `-f` flag; when a hash is selected with `-f`, subsequent hash slots in multi-hash algorithms inherit the selection.
3. **Ordering table** (`-r`): compute a permutation array π such that $\pi[h(k_i)] = i$, mapping each output hash index back to the position of the corresponding key in the input file.
4. **Reordered keyfile** (`-R`): write the input keys in the order defined by the MPHF, i.e., emit key k_i at line $h(k_i)$ of the output, producing a key file sorted by hash value.

This paper is structured as follows. Section 2 reviews the CMPH algorithm suite and its properties. Section 3 describes the new practical features in releases 2.0–2.1. Section 4 discusses the space cost of ordering and the important distinction between ordered and unordered MPHFs. Section 5 places CMPH in the context of the modern MPHF landscape. Section 6 concludes.

2 The CMPH Algorithm Suite

CMPH implements nine algorithm variants, covering the main families of practical MPHF construction. This table summarizes their key properties.

Algorithm	**Space (bits/key)**	**Order-pres.**	**Scalability**	**Reference**
CHM	33 bytes/key[1]	Yes	Internal memory	[6]
BMZ	3.72n bytes	No	Internal memory	[7]
BMZ8	Same, ≤ 256 keys	No	Small sets	[7]
BRZ	8.1 bits/key	No	External, billions	[8]
FCH	4 bits/key	No	Small sets	[9]
BDZ	2.62 bits/key	No	Internal memory	[1]
BDZ_PH	1.95 bits/key	No	PHF only	[1]
CHD_PH	1.40 bits/key	No	PHF only	[4]
CHD	2.07 bits/key	No	Internal memory	[4]

Table 1: CMPH algorithms with approximate space consumption, order-preservation property, and scalability. "PHF only" denotes partial (non-minimal) perfect hash functions.

2.1 BMZ and CHM

The BMZ algorithm [7] uses acyclic random graphs with $c \cdot n$ vertices ($c \approx 0.93$ or 1.15) and generates MPHFs in expected linear time with $3.72n$ bytes storage. CHM [6] uses larger graphs ($c \approx 2.09$) but is the only order-preserving algorithm in CMPH: the output hash value of each key equals its position in the input list. This property comes at a storage cost of $8.36n$ bytes (storing a function value per vertex), compared to BMZ's $3.72n$.

2.2 BDZ

The BDZ algorithm (also called BPZ) [1] uses random 3-partite hypergraphs. Each key is mapped to an edge connecting three vertices. An acyclic hypergraph is constructed in the mapping step, then a function g is assigned and a compact rank structure built. For $m = cn$

[1] CHM stores g as an array of 4-byte integers; the function values are $O(\log n)$ bits per key, i.e., approximately $4cn$ bytes.

with $c \geq 1.23$, the resulting MPHF needs approximately $2.62n$ bits. The PHF variant BDZ_PH omits the rank step and achieves $1.95n$ bits for $m = 1.23n$.

2.3 CHD

The CHD algorithm [4] (Compress, Hash, and Displace) achieves the best space efficiency in CMPH. It splits keys into buckets, computes displacement values to resolve collisions, and compresses the displacement array using Simple Dense Coding (SDC). For load factor $\alpha = 0.99$ and bucket parameter $\lambda = 6$, it stores MPHFs in approximately 2.07 bits per key. The PHF variant CHD_PH achieves 1.40 bits per key at 81% load. As of its publication in 2009, CHD achieved the best practical space efficiency among linear-time constructions.

2.4 BRZ

The BRZ algorithm [8] is designed for external-memory hashing of very large key sets (in the order of billions). It uses a two-level approach with a partitioning step into temporary files, enabling MPHF construction when the key set does not fit in RAM. It requires approximately 8.1 bits per key.

3 New Features in CMPH 2.0–2.1

3.1 Hash Function Selection

Prior to release 2.0, CMPH used Jenkins hash [10] as its default (and often only practical) hash function. Releases 2.0–2.1 add:

- **wyhash** [5]: a modern, fast non-cryptographic hash function well-suited for variable-length strings. It uses multiply-mix operations on 64-bit words and is among the fastest available options for short keys.
- **CRC32**: hardware-accelerated cyclic redundancy check, suitable when hardware intrinsics are available.
- **DJB2**, **FNV**, **SDBM**: classical small string hash functions, retained for compatibility and special cases.

The `-f` flag on the command line selects a hash function for all hash slots. For algorithms requiring multiple independent hash functions (BDZ uses three, CHM/BMZ use two), the same selection is propagated:

```
cmph -g -a chd -f wyhash -m output.mph keys.txt
```

This allows benchmarking different hash functions and choosing the best for a given key set.

3.2 Compilation to Standalone C

The most significant new feature is the `-C` flag, which compiles an MPHF into a standalone C source file:

```
cmph -g -a bdz -C -o mphf.c -p mylib_ keys.txt
```

The output `mphf.c` contains:

1. The hash function (selected by `-f`), inlined as a static function with the seed baked in.
2. The MPHF data arrays as `const` byte or uint32 arrays with a compile-time `#define` for each array's length.
3. A search function with the prefix given by `-p` (default `cmph_c_`):

```c
uint32_t mylib_search(const char *key, uint32_t keylen);
```

The compiled function is entirely self-contained: it links against no external library, allocates no heap memory during lookup, and typically fits in a few hundred bytes to a few kilobytes of object code. This makes it suitable for:

- **Compiler keyword recognition**: keyword tables in programming language front-ends (e.g., reserved words in C, Python, or domain-specific languages) can be compiled in at build time and need no runtime setup.
- **Embedded systems**: constrained environments where dynamic allocation and shared library linking are unavailable.
- **Security-sensitive code**: eliminating runtime library dependencies reduces attack surface.
- **Performance-critical lookup paths**: inlined constants allow the compiler to apply constant-folding and better code generation than a loaded binary MPHF.

Supported algorithms for compilation include BMZ, BDZ (and BDZ_PH), CHD (and CHD_PH), FCH, and CHM. BRZ is excluded as it relies on external temporary files during construction, though the resulting MPHF itself could in principle be compiled.

The C function prefix (`-p`) allows multiple compiled MPHFs to coexist in the same translation unit without name conflicts.

3.3 Ordering Table and Reordered Keyfile

Two new flags address the problem of mapping between hash values and input key positions:

`-r` (ordering table): After constructing the MPHF, compute a permutation array π of size n such that $\pi[h(k_i)] = i$ for all i. That is, $\pi[j]$ gives the line number in the input file of the key whose hash value is j. The ordering table is written to the `.mph` file and can be retrieved via `cmph_ordering_table()`.

`-R` (reordered keyfile): Write a new key file where line j contains the key $k_{\{\pi[j]\}}$, i.e., the keys are listed in ascending order of their hash values. If the MPHF is later used to index a parallel array of values, the reordered keyfile allows constructing that array in the natural hash-value order without a secondary lookup.

These features are especially useful when the MPHF is used as an index into a packed array of associated values. Without them, the caller must maintain the π mapping externally, or accept that values are stored in the order the hash assigns — which is not the input order.

A practical example: given a file of identifiers and an associated file of values at the same line positions, `-R` enables building the value array in hash-index order directly:

```
cmph -g -a chd -r -R keys_reordered.txt -m ids.mph ids.txt
# Now: ids_reordered.txt[j] is the key with hash value j
# Associate values in the same order for direct indexed access
```

4 The Space Cost of Ordering: A Common Misconception

A subtle but practically important point deserves attention: **a minimal perfect hash function without an ordering guarantee is fundamentally smaller than one that preserves key order.**

4.0.1 Non-Ordered MPHFs

Most CMPH algorithms (BMZ, BDZ, CHD, etc.) produce MPHFs that are bijections from S to $[0, n-1]$, but the assignment of hash values to keys is determined by the construction algorithm

and seed — it is essentially random. The information content of such an MPHF is $\log_2 e + o(1) \approx 1.443$ bits per key (the theoretical lower bound for MPHFs [4]).

4.0.2 Order-Preserving MPHFs

An *order-preserving* MPHF additionally requires $h(k_i) = i$ for a given ordering of keys. This means the function must encode a specific permutation of S. Since there are $n!$ possible orderings, storing the order requires at least

$$\log_2 \frac{n!}{n} = \log_2 n - \log_2 e + O\left(\frac{1}{n}\right) \approx \log_2 n - 1.443$$

bits per key by Stirling's approximation. For $n = 10^6$, this is about 19.9 bits per key versus 1.44 bits per key — a factor of almost 14×. The CHM algorithm achieves order preservation at a cost of $8.36n$ bytes (storing 4-byte integers per vertex), which for large n reflects this fundamental overhead.

4.0.3 Ordering Table: A Middle Ground

The ordering table introduced by `-r` offers a middle ground. The MPHF itself remains compact (e.g., CHD at 2.07 bits/key), but an additional array of n 32-bit integers ($32n$ bits $= 4n$ bytes) records the permutation π. The total overhead is $4n$ bytes, which is large compared to the MPHF itself but avoids storing the permutation *inside* the hash function structure.

Important implication for users: If you benchmark an MPHF implementation that claims, for example, 2 bits/key but does not preserve or expose the key order, it is not directly comparable to an order-preserving structure. The "missing" bits are not a trick — they genuinely encode less information. If your application requires knowing which key corresponds to a given hash value (e.g., to look up an associated value by key position), you must either:

1. Use an order-preserving algorithm (CHM in CMPH, at higher cost), or
2. Store the ordering table separately (4 bytes/key extra), or
3. Store key–value pairs indexed by hash value (requires the full key or a fingerprint).

This distinction is sometimes glossed over in benchmarks that compare "bits per key" across different algorithm families without noting whether ordering information is included.

5 CMPH in the Context of Modern MPHF Research

Since the publication of the CHD algorithm [4], the field of minimal perfect hashing has advanced dramatically. We summarize the key developments based on the recent survey by Lehmann et al. [2] and place CMPH algorithms in this context.

5.1 The Theoretical Landscape

The space lower bound for an MPHF is $n \log_2 e - O(\log n) \approx 1.443n$ bits [4]. When the CHD paper was written in 2009, the best practical algorithms achieved about 2.07 bits/key — a factor of 1.43 × above the lower bound. As of 2025, CONSENSUS-RecSplit achieves 1.444 bits/key [11], effectively reaching the lower bound for practical purposes.

Modern approaches fall into three categories [2]:

1. **Perfect hashing through retrieval** (CHM, MWHC/BDZ, ShockHash, MorphisHash): use retrieval data structures to map keys to output values.
2. **Perfect hashing through brute-force** (RecSplit, SIMDRecSplit, CONSENSUS): partition the key set into small subsets and find hash functions by exhaustive search.

3. **Perfect hashing through fingerprinting** (BBHash, FMPH, FiPS): assign small fingerprints and resolve collisions recursively.

5.2 PTHash

PTHash [12] combines the query speed of FCH [9] with the space efficiency of CHD [4]. It uses an imbalanced bucket assignment (60% of keys to 30% of buckets) and compressed displacement values, achieving approximately 2–3 bits/key depending on configuration. A key innovation is flexible encoding (fixed-width, Elias-Fano, dictionary) that allows a single memory access per query. Later variants (PTHash-HEM) support parallel construction. PTHash can be viewed as a direct successor to CHD with better engineering for modern hardware.

The author worked on a compiler for PTHash (github.com/rurban/pthash, the static-hash variant) to emit a C++ function for a given keyset. The straighforward approach of pure nested C++ functions was not possible with the in Januray 2025 non-C26 C++ meta reflection capabilities of the gnu and clang compilers. It did create a proper function, but the compilation produced wrong results. Splitting up the single nested function in subsequent steps of intermediary results produced a correct function, but the author rather wanted to wait for compiler fixes, then rewriting the compiler to emit gensym'ed intermediate results.

5.3 RecSplit

RecSplit [13] was the first approach to significantly close the gap to the theoretical lower bound in practice. It recursively partitions keys into buckets of expected size b (typically 100–2000), and within each bucket builds a binary splitting tree down to small leaves of size ℓ. At each internal node and leaf, brute-force search finds a hash function seed. Seeds are stored using Golomb-Rice coding, approaching the entropy-optimal space. With $b = 2000$ and $\ell = 8$–16, RecSplit achieves approximately 1.6–2 bits/key while maintaining $O(n)$ expected construction time.

5.4 SIMDRecSplit

SIMDRecSplit [14] accelerates RecSplit construction using three levels of parallelism: bit-parallelism via a rotation-fitting technique (testing multiple hash function seeds simultaneously), word-level SIMD instructions, and multi-threading across buckets. GPURecSplit further exploits GPU CUDA streams. SIMDRecSplit achieves the same space as RecSplit but with substantially higher construction throughput, making large-scale use practical.

5.5 ShockHash and Bipartite ShockHash

ShockHash [15] represents an extreme version of the retrieval-based approach. Each key corresponds to an edge between two candidate output values; an MPHF corresponds to an orientation of the resulting graph (each node has in-degree 1). A valid orientation exists if and only if the graph is a pseudoforest. ShockHash uses brute-force over hash function seeds until a pseudoforest is found, then stores the 1-bit orientation choices in a retrieval data structure (n bits) and the seed in $0.44n$ bits. Combined with RecSplit-style partitioning for large sets, ShockHash achieves 1.52 bits/key at construction costs comparable to RecSplit at 1.6 bits/key.

Bipartite ShockHash [15] stores two independent seeds (one per endpoint of each edge), building seed pools and testing combinations. The bipartite variant achieves 1.489 bits/key. The ShockHash-Flat variant uses a k-perfect hash function for partitioning instead of RecSplit, gaining faster queries by avoiding splitting-tree traversal.

5.6 MorphisHash

MorphisHash [16] refines ShockHash by addressing the redundancy in pseudoforest orientation. Each connected component's cycle can be oriented in two ways, contributing 1 bit of redundancy per component. MorphisHash formulates orientation as a system of linear equations modulo 2 and stores its solution in a compact retrieval data structure, eliminating the cycle- orientation redundancy. It achieves slightly better space than ShockHash and better cache locality during queries, because the retrieval data structures are stored alongside the seeds.

5.7 CONSENSUS

CONSENSUS-RecSplit [11] introduces a fundamentally new seed-encoding strategy. Instead of encoding each subtask's seed independently with Golomb-Rice, CONSENSUS gives each subtask a fixed budget of c choices slightly above the expected number needed. If a subtask exhausts its choices, the algorithm *backtracks* to a previous subtask and tries another seed; crucially, the hash function considers the concatenation of all previous seeds, so backtracking provides a genuinely fresh opportunity for the current subtask. This mechanism eliminates the per-subtask coding overhead of variable-length codes.

CONSENSUS-RecSplit uses RecSplit with leaf size $\ell = 1$ (each leaf is a single key). With Golomb-Rice coding, $\ell = 1$ would be extremely wasteful, but CONSENSUS handles it efficiently. The result is the first practical construction to achieve $(1+\varepsilon)n\log_2 e$ bits in $O\left(\frac{n}{\varepsilon}\right)$ expected construction time, versus RecSplit's $O\left(n \cdot e^{\{\frac{1}{\varepsilon}\}}\right)$. A practical configuration achieves 1.444 bits/key — within 0.1% of the theoretical lower bound $\log_2 e \approx 1.443$ bits/key — while construction remains feasible.

The query time of CONSENSUS-RecSplit is approximately 60% slower than RecSplit's, due to the tree traversal and combined seed computation at each level.

5.8 Summary Comparison

This table places CMPH algorithms alongside the modern approaches described above.

Algorithm	**Space (bits/key)**	**Construction**	**Query accesses**	**Year**
CHM (CMPH)	33 bytes	O(n)	2	1992
BMZ (CMPH)	3.72 bytes	O(n)	2	2004
BDZ (CMPH)	2.62	O(n)	3	2007
CHD (CMPH)	2.07	O(n)	1–2	2009
FCH (CMPH)	4	O(n)	1	1992
PTHash	2.0–3.0	O(n)	1	2021
RecSplit	1.6–2.0	O(n)	O(log n)	2020
SIMDRecSplit	1.6–2.0	O(n), fast	O(log n)	2023
ShockHash	1.52	O(n)	O(log n)	2023
Bip. ShockHash-RS	1.489	O(n)	O(log n)	2023
MorphisHash	1.47–1.49	O(n)	O(log n)	2025
CONSENSUS-RS	1.444	$O\left(\frac{n}{\varepsilon}\right)$	O(log n)	2025

Table 2: Approximate space, construction complexity, query memory accesses, and year for CMPH algorithms and leading modern approaches. Space for CMPH legacy algorithms (CHM, BMZ) is dominated by storing $O(\log n)$-bit integer arrays per key, not just the hash function description.

5.9 Where CMPH Fits

CMPH's CHD algorithm remains competitive with PTHash in both space (2.07 bits/key) and construction time. For applications requiring the absolute minimum space, CMPH should be complemented by or replaced with RecSplit, ShockHash, or CONSENSUS-RecSplit depending on the acceptable query time.

However, CMPH occupies a distinct niche: it is a **portable C library** with no C++ or SIMD requirements, supporting external-memory construction (BRZ), t-perfect hashing (CHD with $t > 1$), and now C code compilation. These practical properties matter in constrained or heterogeneous environments where cutting-edge but architecture-specific C++ libraries may not be suitable.

6 Benchmarks

We measured construction and lookup performance on datasets of $n = 10^6$ keys, compiled with `-DNDEBUG`. Two key types are tested: 4-byte integer keys (Table 3) and random URL-like strings of 40–80 characters (Table 4). Construction time is nanoseconds per key (total / n). Lookup time is nanoseconds per query over $100n = 10^8$ random lookups. C file sizes are for 1M keys and scale linearly with n: for $n \approx 200$ keywords (a typical compiler keyword table), all compiled outputs fit in a few kilobytes.

The benchmark programs are `bm_numbers` / `bm_strings` (runtime library lookup via `cmph_search`) and `bm_compiled` / `bm_cstrings` (compiled C function, loaded with `dlopen`, no library linkage). The speedup column is the ratio of library to compiled lookup time.

Table 3 shows integer key results for all algorithms across Jenkins, wyhash, FNV, and CRC32.

Algo	Hash	C size (KB)	Create (ns/key)	Lib (ns)	Compiled (ns)	Speedup
BMZ	Jenkins	19,673	1,486	350	283	1.24×
BMZ	wyhash	19,668	1,534	340	270	1.26×
CHM	Jenkins	21,105	1,412	360	286	1.26×
CHM	wyhash	21,104	1,390	346	269	1.29×
BDZ	Jenkins	6,509	695	392	192	2.04×
BDZ	wyhash	6,509	692	382	184	2.08×
BDZ	FNV	6,508	712	407	192	2.12×
BDZ	CRC32	6,511	655	361	180	2.01×
BDZ_PH	Jenkins	3,323	705	318	163	1.95×
BDZ_PH	wyhash	3,322	691	305	156	1.96×
BDZ_PH	FNV	3,321	748	325	167	1.95×
BDZ_PH	CRC32	3,325	671	291	153	1.90×
FCH	Jenkins	1,828	15,862	372	160	2.33×
FCH	wyhash	1,828	20,030	355	141	2.52×
CHD_PH	Jenkins	1,420	1,291	429	190	2.26×
CHD_PH	wyhash	1,419	1,272	425	186	2.28×
CHD_PH	FNV	1,418	1,303	436	196	2.22×
CHD_PH	CRC32	1,426	1,258	400	186	2.15×
CHD	Jenkins	807	1,293	600	246	2.44×
CHD	wyhash	806	1,277	602	237	2.54×
CHD	FNV	806	1,331	631	245	2.57×
CHD	CRC32	808	1,263	559	237	2.36×

Table 3: Construction and lookup performance, $n = 10^6$ keys, NDEBUG build. Lookup is ns/query over 10^8 random lookups. C size scales linearly with n (divide by 1000 for $n = 1000$). FCH construction is slow due to exhaustive search but yields the smallest compiled output. CHD achieves the highest compiled speedup because constant-folding the SDC displacement array is especially effective.

Key observations:

- **CHD benefits most from compilation**: CHD library lookup is the slowest among MPHF algorithms (559–631 ns depending on hash), but compiled lookup drops to 237–246 ns — a **2.4–2.6x speedup**. This is because CHD's SDC-compressed displacement array is an ideal target for compiler constant-folding when baked into a `const` C array. After compilation, CHD is faster than library BDZ.
- **BDZ and BDZ_PH gain 2x**: Both roughly double in lookup speed under compilation (BDZ: 361–407 ns → 180–192 ns; BDZ_PH: 291–325 ns → 153–167 ns), making them competitive with FCH without FCH's expensive construction.
- **FCH: fastest lookup, smallest file**: FCH compiled with wyhash reaches 141 ns/query (the best overall), and at 1.8 MB per 1M keys it produces by far the smallest compiled output. At $n = 1000$ the file is under 2 KB. The tradeoff is construction time of 15–20 s per 1M keys, acceptable only for static, small key sets built offline.
- **CHD C file is the most compact for large n**: At 807 KB for 1M keys, CHD's compiled output is 8x smaller than BDZ (6.5 MB) and 24x smaller than BMZ (19.7 MB), because CHD stores displacement values in compressed form rather than full arrays.
- **wyhash is consistently fastest**: wyhash reduces lookup latency by 3–10 ns over Jenkins for both library and compiled variants. CRC32 is a close second and also competitive.

- **BMZ and CHM gain least**: Both show only a 1.24–1.29x speedup from compilation. Their compiled files are also the largest (20 MB at 1M keys), making them poor candidates for compiled deployment at scale.

6.0.1 String Key Benchmarks

The preceding benchmarks use 4-byte integer keys, where the hash computation is fast and the lookup table dominates query time. To measure a more realistic workload, we benchmarked the same algorithms on 10^6 random URL-like strings of 40–80 characters (average 60 bytes), typical of web-scale key sets.

Table 4 shows results for library and compiled lookup across Jenkins, wyhash, and FNV hash functions. SDBM and CRC32 failed to produce valid MPHFs for this key set.

Algo	**Hash**	**Create (ns/key)**	**Lib (ns)**	**Compiled (ns)**	**Speedup**
BMZ	Jenkins	1,001	463	448	1.03×
CHM	Jenkins	1,327	485	448	1.08×
BDZ	Jenkins	427	480	435	1.10×
FCH	Jenkins	12,601	464	439	1.06×
BDZ_PH	Jenkins	444	378	376	1.01×
CHD_PH	Jenkins	581	419	381	1.10×
CHD	Jenkins	579	468	425	1.10×
BMZ	wyhash	912	413	406	1.02×
CHM	wyhash	850	413	409	1.01×
BDZ	wyhash	382	405	402	1.01×
FCH	wyhash	8,095	346	404	0.86×
BDZ_PH	wyhash	379	356	378	0.94×
CHD_PH	wyhash	535	364	342	1.06×
CHD	wyhash	523	425	399	1.07×
BMZ	FNV	1,224	717	533	1.35×
CHM	FNV	1,158	719	482	1.49×
BDZ	FNV	674	703	587	1.20×
FCH	FNV	10,800	544	459	1.19×
BDZ_PH	FNV	694	645	527	1.22×
CHD_PH	FNV	830	639	518	1.23×
CHD	FNV	811	695	555	1.25×

Table 4: String key benchmarks: $n = 10^6$ random URL-like strings (40–80 chars). Create is ns/key (library). Lookup is ns/query over 10^8 random lookups. Compiled C sizes are identical to Table 3 (they depend on n, not key length). DJB2 is omitted for brevity (similar to FNV). SDBM and CRC32 failed on this key set.

Key observations for string keys:

- **Compilation speedup largely disappears.** With 4-byte integer keys (Table 3), compiled lookup was 1.2–2.6× faster than the library. With 60-byte strings, the speedup drops to 1.0–1.1× for Jenkins and wyhash, because the hash computation over the key bytes now dominates query time and is identical in both paths. The compiled lookup tables provide little benefit when hashing itself is the bottleneck.

- **wyhash with compilation can be *slower***: for FCH and BDZ_PH with wyhash, the compiled variant is slightly slower than the library (0.86× and 0.94×), likely due to code-cache pressure from the large compiled arrays offsetting any table-access savings.

- **FNV shows the largest compiled speedup on strings** (1.2–1.5×). FNV's byte-at-a-time loop is slower than Jenkins or wyhash for longer keys, making the lookup table contribution relatively more significant. CHM+FNV compiled achieves 1.49× — the highest string-key speedup — because CHM's large lookup array benefits most from constant-folding.
- **wyhash remains the fastest hash for strings**: FCH+wyhash library achieves 346 ns/query, the fastest overall string lookup. BDZ_PH+wyhash at 356 ns is a close second with much faster construction (379 vs 8,095 ns/key).
- **Practical recommendation**: for compiled deployment with string keys, prefer BDZ_PH or CHD_PH with wyhash — they offer the best balance of fast lookup, compact compiled output, and fast construction. Compilation is primarily valuable for integer or short keys where table access dominates; for URL-length strings, the library is nearly as fast.

7 Implementation Notes

7.1 BRZ and Ordering

The BRZ algorithm [8] required special handling to support the `-r` ordering table. BRZ operates by partitioning keys into temporary buckets on disk, constructing local MPHFs per bucket, and combining them. The ordering table must track positions across bucket boundaries. Release 2.1 extends BRZ to support `-r` and `-R` by maintaining global position indices during the merging phase.

7.2 Compilation Architecture

The C compilation path (`cmph_compile`) works as follows:

1. The MPHF is constructed and stored in the internal `cmph_t` structure as usual.
2. `cmph_compile()` dispatches to an algorithm-specific compile function (e.g., `bdz_compile()`, `chd_compile()`).
3. Each algorithm emits its internal arrays via `bytes_compile()` and `uint32_compile()` helpers, which write initialized C array definitions with `const` qualifiers.
4. The hash seed (from `cmph_config_t.seed`) is written as a compile-time constant.
5. A `cmph_c_search()` (or prefixed variant) function is emitted, implementing the lookup using only the emitted arrays and the inlined hash function.

The C function prefix (`-p`) defaults to `cmph_c_` but is configurable to allow coexistence of multiple compiled MPHFs in the same compilation unit.

For algorithms using the `hash_vector` optimization (BMZ, CHM), compile support requires special care to inline the vector initialization; all such paths have been implemented and tested.

7.3 Multiple Hash Function Support

Several correctness issues were uncovered and fixed in release 2.0.3: The broken algorithms `bmz8`, `chd`, `chd_ph` were fixed. Memory leaks and heap-overflows were fixed. Wrong hash states for non-Jenkins hash functions, causing incorrect behavior with `bdz`, `chm`, `chd`, and `chd_pc`. wyhash [5], and CRC (SW and HW) were added as a new default alternatives with competitive speed.

8 Conclusion

CMPH 2.0 and 2.1 extend a mature, portable MPHF library with practical tools addressing real deployment needs: compile-time embedding via C code generation, multiple hash function choices including the modern wyhash, and the ability to produce ordering tables or reordered

key files. A clarifying observation is that a compact MPHF's small size is *not* a deficiency but reflects the minimal information needed to represent a bijection without ordering; adding an ordering guarantee requires $O(n \log n)$ bits fundamentally.

In the broader research landscape, CMPH's CHD algorithm remains practical and competitive with PTHash, while newer approaches (RecSplit, ShockHash, MorphisHash, CONSENSUS) push further toward the theoretical minimum of $\log_2 e \approx 1.443$ bits/key. CONSENSUS-RecSplit [11] achieves 1.444 bits/key in $O\left(\frac{n}{\varepsilon}\right)$ time, effectively closing the gap to the lower bound. The trade-off is query time: all RecSplit-family algorithms traverse a splitting tree, giving $O(\log n)$ memory accesses, while CHD and PTHash achieve 1–2 accesses.

Future directions for CMPH include: adding support for 64-bit key counts (currently limited to 32-bit), implementing a fingerprinting-based algorithm (BBHash or FMPH) for medium space efficiency with simple parallel construction, and exploring GPU compilation output alongside C.

Bibliography

[1] F. C. Botelho, "Near-Optimal Space Perfect Hashing Algorithms," Doctoral dissertation, 2008.

[2] H.-P. Lehmann *et al.*, "Modern Minimal Perfect Hashing: A Survey." 2025.

[3] H.-P. Lehmann, P. Sanders, and S. Walzer, "Combined Search and Encoding for Seeds, with an Application to Minimal Perfect Hashing." 2025. doi: 10.48550/ARXIV.2502.05613.

[4] D. Belazzougui, F. C. Botelho, and M. Dietzfelbinger, "Hash, displace, and compress," in *Proceedings of the 17th European Symposium on Algorithms (ESA'09)*, in Lecture Notes in Computer Science. Springer, 2009.

[5] Y. Wang, "wyhash: A Fast Portable Non-Cryptographic Hashing Algorithm." 2019.

[6] Z. J. Czech, G. Havas, and B. S. Majewski, "An Optimal Algorithm for Generating Minimal Perfect Hash Functions," in *Information Processing Letters*, 1992, pp. 257–264.

[7] F. C. Botelho, R. Pagh, and N. Ziviani, "Simple and Space-Efficient Minimal Perfect Hash Functions," in *Proceedings of the 10th Workshop on Algorithms and Data Structures (WADS'07)*, in Lecture Notes in Computer Science, vol. 4619. Springer, 2007, pp. 139–150.

[8] F. C. Botelho, Y. Kohayakawa, and N. Ziviani, "A Practical Minimal Perfect Hashing Method," in *Proceedings of the 4th International Workshop on Efficient and Experimental Algorithms (WEA'05)*, in Lecture Notes in Computer Science, vol. 3503. Springer, 2005, pp. 488–500.

[9] E. A. Fox, Q. F. Chen, and L. S. Heath, "A faster algorithm for constructing minimal perfect hash functions," in *Proceedings of the 15th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, Copenhagen, Denmark: ACM, 1992, pp. 266–273.

[10] B. Jenkins, "Hash Functions." 1997.

[11] H.-P. Lehmann, P. Sanders, and S. Walzer, "CONSENSUS: Brute-Force Perfect Hashing without Compromise." 2025.

[12] G. E. Pibiri and R. Trani, "PTHash: Revisiting FCH Minimal Perfect Hashing," in *Proceedings of the 44th ACM SIGIR Conference on Research and Development in Information Retrieval*, ACM, 2021, pp. 1339–1348.

[13] E. Esposito, T. M. Graf, and S. Vigna, "RecSplit: Minimal Perfect Hashing via Recursive Splitting," in *Proceedings of the 22nd Workshop on Algorithm Engineering and Experiments (ALENEX)*, SIAM, 2020, pp. 175–185.

[14] H.-P. Lehmann, P. Sanders, and S. Walzer, "SIMDRecSplit: Vectorized Minimal Perfect Hashing," in *Proceedings of the 25th Workshop on Algorithm Engineering and Experiments (ALENEX)*, SIAM, 2023.

[15] H.-P. Lehmann, P. Sanders, and S. Walzer, "ShockHash: Towards Optimal-Space Perfect Hashing With Linear Construction Time," in *Proceedings of the 25th Workshop on Algorithm Engineering and Experiments (ALENEX)*, SIAM, 2023.

[16] D. Gog, H.-P. Lehmann, J. Mattes, P. Sanders, and S. Walzer, "MorphisHash: Approaching the Space Optimum for Minimal Perfect Hashing." 2025.